

Sprawozdanie z Laboratorium

Obliczenia Naukowe - Lista 5

Wojciech Typer

6 stycznia 2026

Zadanie 1

Cel zadania

Celem zadania jest napisanie funkcji rozwiązującej układ $ax = b$ metodą eliminacji Gaussa, uwzględniając specyficzną postać macierzy A , która jest macierzą rzadką:

$$A = \begin{pmatrix} A_1 & C_1 & 0 & 0 & 0 & \dots & 0 \\ B_2 & A_2 & C_2 & 0 & 0 & \dots & 0 \\ 0 & B_3 & A_3 & C_3 & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & B_{v-2} & A_{v-2} & C_{v-2} & 0 \\ 0 & \dots & 0 & 0 & B_{v-1} & A_{v-1} & C_{v-1} \\ 0 & \dots & 0 & 0 & 0 & B_v & A_v \end{pmatrix},$$

dla dwóch wariantów:

- bez wyboru elementu głównego
- z częściowym wyborem elementu głównego

Funkcje pomocnicze

- `lu_dense!` – wykonuje dekompozycję LU macierzy gęstej bez wyboru elementu głównego.
- `lu_dense_pivot!` – wykonuje dekompozycję LU macierzy gęstej z częściowym wyborem elementu głównego.
- `solve_l!` – realizuje podstawienie w przód ($Ly = b$) dla macierzy dolnotrójkątnej, uwzględniając opcjonalną permutację wektora b .
- `solve_u!` – realizuje podstawienie wsteczne ($Ux = y$) dla macierzy górnortrójkątnej.
- `compute_W_matrix` – oblicza macierz pomocniczą W , definiowaną jako $W = A_k^{-1}C_k$, gdzie C_k to macierz diagonalna.
- `mul_sub_B_W!` – wykonuje aktualizację kolejnego bloku diagonalnego według wzoru $A_{k+1} = A_{k+1} - B_k \cdot W$.

Struktura danych

Do zaimplementowania algorytmu użyto następującej struktury danych:

Algorithm 1 Definicja struktur danych macierzy blokowej

```
1: struct BorderMatrix
2:   l: rozmiar bloku
3:   top_row: wektor przechowujący pierwszy wiersz bloku
4:   right_column: wektor przechowujący ostatnią kolumnę bloku
5: end struct
6:
7: struct CMatrix
8:   l: rozmiar bloku
9:   diag: wektor przechowujący przekątną bloku
10: end struct
11:
12: struct BlockMatrix
13:   l: rozmiar bloku  $l \times l$ 
14:   v: liczba bloków w macierzy ( $n = v \cdot l$ )
15:   A: wektor gęstych bloków diagonalnych
16:   B: wektor bloków rzadkich typu BorderMatrix
17:   C: wektor bloków rzadkich typu CMatrix
18: end struct
```

BorderMatrix – przechowuje rzadki blok znajdujący się pod diagonalą.

- *top_row*: zawiera pierwszy wiersz bloku, odpowiedzialny za powiązanie z poprzednim blokiem diagonalnym.
- *right_column*: przechowuje ostatnią kolumnę, istotną przy redukcji elementu brzegowego.

CMatrix – przechowuje rzadki blok znajdujący się nad diagonalą.

- *diag*: zawiera tylko elementy na przekątnej bloku, co redukuje pamięć z $O(l^2)$ do $O(l)$.

BlockMatrix – główny kontener macierzy, zapewniający liniową złożoność $O(n)$.

- *l*, *v*: wymiary określające strukturę blokową macierzy.
- *A*: wektor macierzy gęstych, na których wykonywana jest dekompozycja LU.
- *B*, *C*: wektory bloków rzadkich, które minimalizują liczbę operacji na zerach.

Implementacja

Idea algorytmu eliminacji Gaussa bez wyboru elementu głównego

Algorytm jest wariantem eliminacji Gaussa dostosowanym do macierzy o strukturze blokowo-trójdzielnej. Wykorzystuje on fakt, że niezerowe elementy macierzy występują wyłącznie w blokach diagonalnych oraz w ich bezpośrednim sąsiedztwie. Dzięki temu eliminacja wykonywana jest jedynie na niewielkich blokach macierzy, a nie na całej macierzy jednocześnie.

Na każdym etapie przeprowadzany jest rozkład LU bloku diagonalnego, bez dokonywania zamian wierszy. Uzyskane czynniki służą do eliminacji wpływu sąsiednich bloków oraz do modyfikacji prawej strony układu. Takie podejście pozwala znacząco ograniczyć liczbę operacji obliczeniowych w porównaniu z klasyczną eliminacją Gaussa. Rozwiązanie końcowe wyznaczane jest w etapie podstawienia wstecznego, również z wykorzystaniem struktury blokowej macierzy.

Algorithm 2 Eliminacja Gaussa dla macierzy blokowej (Zadanie 1a)

```
1: procedure SOLVEBLOCKGAUSS( $v, l, A, B, C, b$ )
2:    $x \leftarrow \text{copy}(b)$ 
3:    $As \leftarrow \text{copy}(A)$ 
4:
5:   for  $k \leftarrow 1$  to  $v$  do ▷ ETAP 1: Eliminacja w przód
6:     LU_Dense( $As_k$ )
7:      $start \leftarrow (k - 1) \cdot l + 1$ 
8:      $end \leftarrow k \cdot l$ 
9:     Solve_L( $As_k, x_{start:end}$ )
10:    if  $k < v$  then
11:       $W \leftarrow \text{Compute\_W}(As_k, C_k)$ 
12:       $As_{k+1} \leftarrow As_{k+1} - B_k \cdot W$ 
13:       $z \leftarrow \text{copy}(x_{start:end})$ 
14:      Solve_U( $As_k, z$ )
15:       $ns \leftarrow k \cdot l + 1$ 
16:       $ne \leftarrow (k + 1) \cdot l$ 
17:       $x_{ns:ne} \leftarrow x_{ns:ne} - B_k \cdot z$ 
18:    end if
19:  end for
20: ▷ ETAP 2: Podstawienie wsteczne
21:  Solve_U( $As_v, x_{(v-1)l+1:vl}$ )
22:  for  $k \leftarrow v - 1$  down to  $1$  do
23:     $r\_k \leftarrow (k - 1)l + 1 : kl$ 
24:     $r\_n \leftarrow kl + 1 : (k + 1)l$ 
25:     $corr \leftarrow C_k \cdot x_{r\_n}$ 
26:    Solve_L( $As_k, corr$ )
27:     $x_{r\_k} \leftarrow x_{r\_k} - corr$ 
28:    Solve_U( $As_k, x_{r\_k}$ )
29:  end for
30:  return  $x$ 
31: end procedure
```

Analiza złożoności obliczeniowej:

- **Złożoność czasowa:** Każdy blok diagonalny wymaga $O(l^3)$ operacji na dekompozycję LU oraz $O(l^2)$ operacji na aktualizację sąsiednich bloków. Dla v bloków, całkowita złożoność czasowa wynosi $O(v \cdot l^3) = O(n \cdot l^2)$. Dla małych, stałych wartości l , złożoność upraszcza się do $O(n)$.

Idea eliminacji Gaussa z częściowym wyborem elementu głównego

Wersja algorytmu z częściowym wyborem elementu głównego poprawia stabilność numeryczną obliczeń. Na każdym etapie eliminacji, osobno dla każdego bloku diagonalnego, wybierany jest element o największej wartości bezwzględnej w aktualnej kolumnie, który następnie trafia na przekątną poprzez zamianę wierszy wewnątrz bloku.

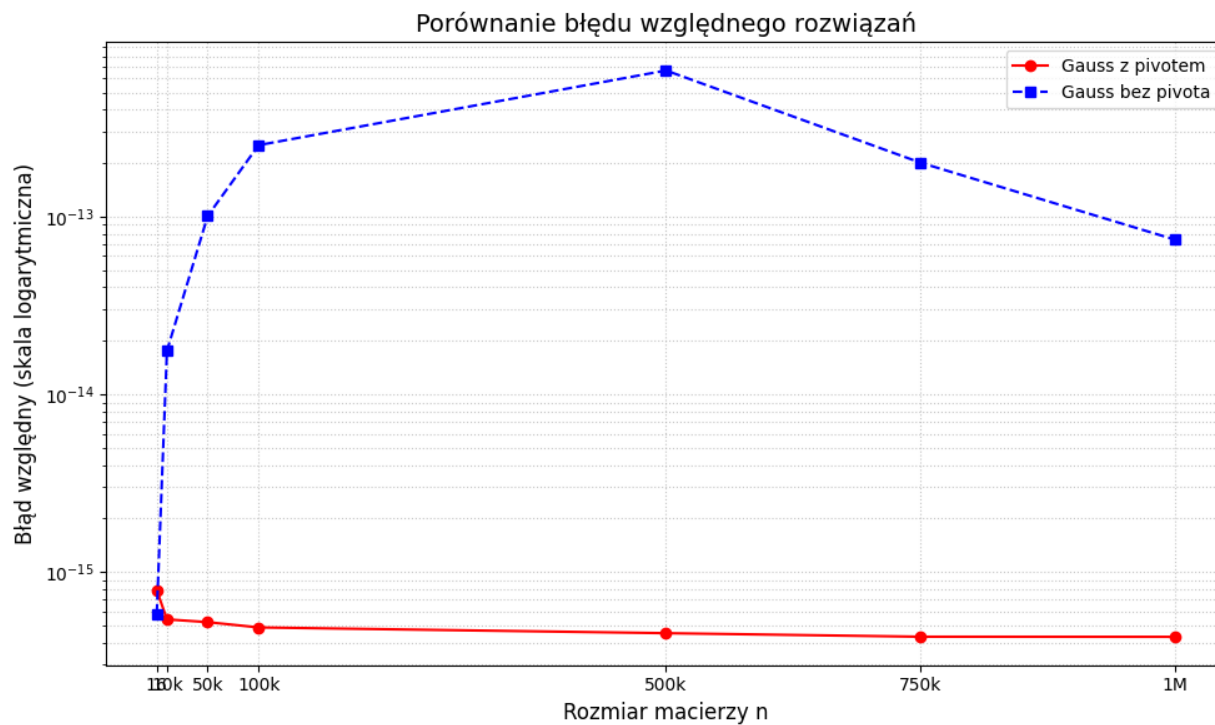
Informacja o wykonanych zamianach wierszy jest zapamiętywana w postaci permutacji i wykorzystywana podczas rozwiązywania układów trójkątnych oraz w etapie podstawienia wstecznego. Takie podejście pozwala uniknąć dzielenia przez bardzo małe liczby, ogranicza wpływ błędów zaokrągleń i znacząco poprawia dokładność wyników, zachowując jednocześnie blokową strukturę macierzy.

Algorithm 3 Eliminacja Gaussa dla macierzy blokowej z częściowym wyborem elementu głównego

```
1: procedure SOLVEBLOCKGAUSSPIVOT( $v, l, A, B, C, b$ )
2:    $x \leftarrow \text{copy}(b)$ 
3:    $As \leftarrow \text{copy}(A)$ 
4:    $P \leftarrow$  array of permutations
5:                                                                  $\triangleright$  ETAP 1: Eliminacja w przód
6:   for  $k \leftarrow 1$  to  $v$  do
7:      $(As_k, P_k) \leftarrow \text{LU\_Dense\_Pivot}(As_k)$ 
8:      $r_k \leftarrow (k-1)l + 1 : kl$ 
9:      $\text{Solve\_L}(As_k, x_{r_k}, P_k)$ 
10:    if  $k < v$  then
11:       $W \leftarrow \text{Compute\_W}(As_k, C_k, P_k)$ 
12:       $\text{UpdateBlock}(As_{k+1}, B_k, W)$ 
13:       $z \leftarrow \text{copy}(x_{r_k})$ 
14:       $\text{Solve\_U}(As_k, z)$ 
15:       $r_n \leftarrow kl + 1 : (k+1)l$ 
16:       $\text{UpdateRHS}(x_{r_n}, B_k, z)$ 
17:    end if
18:  end for
19:                                                                  $\triangleright$  ETAP 2: Podstawienie wsteczne
20:   $\text{Solve\_U}(As_v, x_{(v-1)l+1:vl})$ 
21:  for  $k \leftarrow v-1$  down to  $1$  do
22:     $r_k \leftarrow (k-1)l + 1 : kl$ 
23:     $r_n \leftarrow kl + 1 : (k+1)l$ 
24:     $corr \leftarrow C_k \cdot x_{r_n}$ 
25:     $corr \leftarrow \text{permute}(corr, P_k)$ 
26:     $\text{Solve\_L}(As_k, corr)$ 
27:     $x_{r_k} \leftarrow x_{r_k} - corr$ 
28:     $\text{Solve\_U}(As_k, x_{r_k})$ 
29:  end for
30:  return  $x$ 
31: end procedure
```

Analiza złożoności obliczeniowej:

- **Złożoność czasowa:** Dodatkowy koszt związany z wyszukiwaniem elementu głównego w każdym bloku oraz zastosowaniem permutacji podczas rozwiązywania układu nie wpływa znacząco na złożoność czasową. Operacje na blokach są ograniczone do sąsiednich bloków diagonalnych, więc złożoność czasowa pozostaje $O(v \cdot l^3) \sim O(n \cdot l^2)$, choć z większymi stałymi ukrytymi w notacji O w porównaniu do wariantu bez pivotowania.



Wnioski

Analiza błędów względnych pokazuje, że metoda eliminacji Gaussa z częściowym wyborem elementu głównego jest znacznie bardziej stabilna numerycznie. Dzieje się tak, ponieważ w każdym kroku wybierany jest największy element w kolumnie bloku, co minimalizuje ryzyko dzielenia przez bardzo małe liczby i ogranicza błędy zaokrągleń. Natomiast metoda bez wyboru elementu głównego wykazuje większe błędy przy rozwiązywaniu układów macierzy gorzej uwarunkowanych, co potwierdza jej podatność na niestabilność numeryczną.

Zadanie 2

Cel zadania

Celem zadania jest napisanie funkcji wyznaczających rozkład LU macierzy A metodą eliminacji Gaussa, uwzględniając specyficzną postać macierzy A , która jest macierzą rzadką (taka sama jak w zadaniu 1), dla dwóch wariantów:

- bez wyboru elementu głównego
- z częściowym wyborem elementu głównego

Analiza złożoności obliczeniowej

LU bez wyboru elementu głównego

Pętla zewnętrzna algorytmu przebiega po wszystkich blokach diagonalnych $k = 1, \dots, v$, gdzie $v = n/l$. W każdym bloku wykonywany jest rozkład LU o złożoności $O(l^3)$ dla bloku diagonalnego A_k . Suma wszystkich operacji dla wszystkich bloków daje całkowitą złożoność czasową:

$$O(v \cdot l^3) = O\left(\frac{n}{l} \cdot l^3\right) = O(n \cdot l^2).$$

Dla małych, stałych wartości l , złożoność upraszcza się do $O(n)$, co oznacza praktycznie liniowy czas działania względem rozmiaru macierzy n .

Blokowa LU z częściowym wyborem elementu głównego

W tym wariantcie, dla każdego bloku diagonalnego, wybierany jest element o największej wartości bezwzględnej w kolumnach bloku i zamieniany na przekątną poprzez permutację wierszy wewnątrz bloku. Dzięki temu unikamy dzielenia przez bardzo małe liczby i poprawiamy stabilność numeryczną.

Koszt dodatkowy polega na wyszukaniu elementu głównego w obrębie każdego bloku oraz zastosowaniu permutacji podczas rozwiązywania układu. Operacje na blokach są ograniczone do sąsiednich bloków diagonalnych, więc złożoność czasowa pozostaje w przybliżeniu $O(v \cdot l^3) \sim O(n \cdot l^2)$, choć z większymi stałymi ukrytymi w notacji O w porównaniu do wariantu bez pivotowania. Złożoność pamięciowa pozostaje $O(v \cdot l^2) \sim O(n \cdot l)$, ponieważ przechowujemy wyłącznie bloki diagonalne i graniczne, a nie pełną macierz.

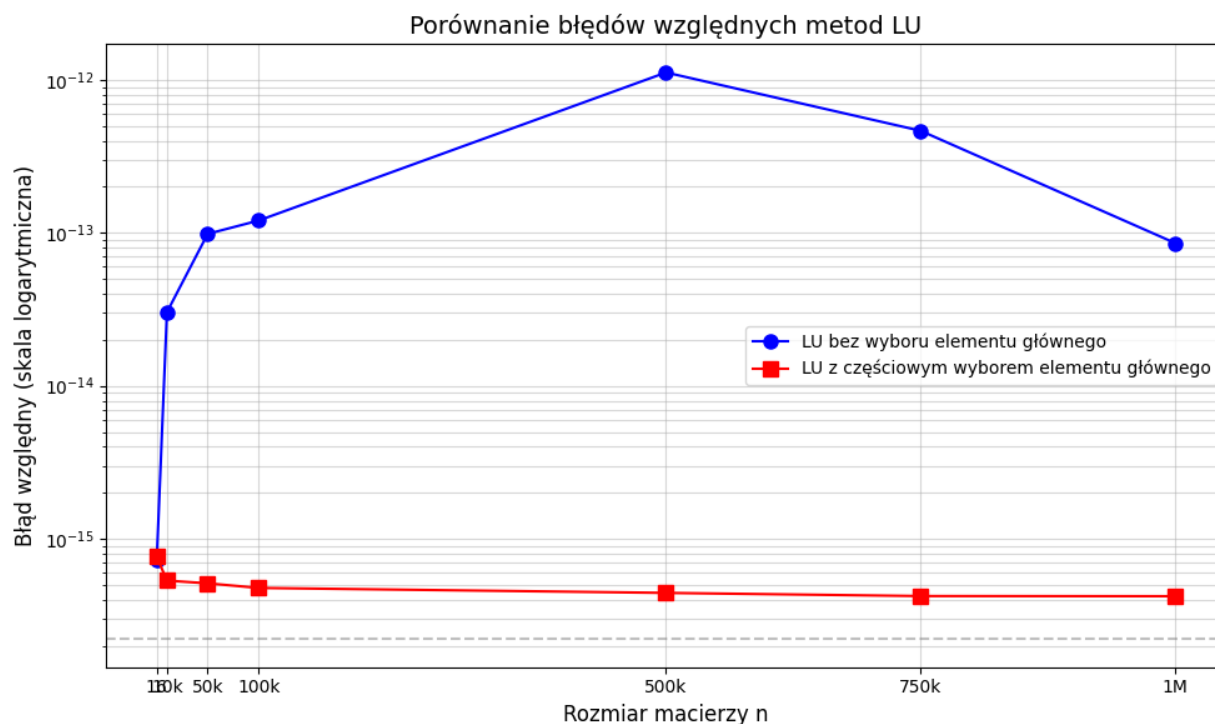
Implementacja

Algorithm 4 Blokowa dekompozycja LU bez wyboru elementu głównego

```
1: procedure BLOCKLU( $v, l, A, B, C$ )
2:   for  $k \leftarrow 1$  to  $v$  do
3:                                     ▷ Rozkład LU bloku diagonalnego
4:     LU_Dense( $A_k$ )
5:     if  $k < v$  then
6:                                     ▷ Obliczenie macierzy pomocniczej
7:        $W \leftarrow \text{Compute\_W}(A_k, C_k)$ 
8:                                     ▷ Aktualizacja następnego bloku diagonalnego
9:        $A_{k+1} \leftarrow A_{k+1} - B_k \cdot W$ 
10:    end if
11:  end for
12: end procedure
```

Algorithm 5 Blokowa dekompozycja LU z częściowym wyborem elementu głównego

```
1: procedure BLOCKLU_PIVOT( $v, l, A, B, C$ )  
2:   perms  $\leftarrow$  pusty wektor długości  $v$  ▷ Przechowuje permutacje dla każdego bloku  
3:   for  $k \leftarrow 1$  to  $v$  do  
4:     ▷ Rozkład LU bloku diagonalnego z częściowym wyborem elementu głównego  
5:     perms[ $k$ ]  $\leftarrow$  LU_Dense_Pivot( $A_k$ ) ▷ Zamiana wierszy w bloku i zapis permutacji  
6:     if  $k < v$  then  
7:       ▷ Obliczenie macierzy pomocniczej z uwzględnieniem permutacji  
8:        $W \leftarrow$  Compute_W( $A_k, C_k, \text{perms}[k]$ )  
9:       ▷ Aktualizacja następnego bloku diagonalnego  
10:       $A_{k+1} \leftarrow A_{k+1} - B_k \cdot W$   
11:    end if  
12:  end for  
13:  return perms ▷ Permutacje są potrzebne przy rozwiązywaniu układu  
14: end procedure
```



Wnioski

Oba warianty algorytmu LU wykazują liniową złożoność czasową $O(n)$ względem rozmiaru macierzy. Metoda z częściowym wyborem elementu głównego jest wolniejsza ze względu na narzut związany z wyszukiwaniem pivota oraz operacje na rozszerzonym pasmie ($2l$ zamiast l). Zapewnia ona jednak znacznie wyższą stabilność numeryczną, utrzymując błąd względny na stałym poziomie precyzji maszynowej (10^{-16}), podczas gdy w wariancie bez pivota błąd ten narasta wraz z rozmiarem macierzy do poziomu 10^{-12} .

Zadanie 3

Cel zadania

Celem zadania jest napisanie funkcji rozwiązujących układ równań $Ax = b$ wykorzystując rozkład LU z zadania 2, uwzględniając specyficzną postać macierzy A , która jest macierzą rzadką (taka sama jak w zadaniu 1).

Złożoność obliczeniowa

Podstawienie w przód

Pętla zewnętrzna przebiega po wszystkich blokach diagonalnych $k = 1, \dots, v$, a każdy blok ma rozmiar l . Dla każdego bloku wykonywane jest podstawianie w przód w lokalnym układzie trójkątnym L oraz korekta wartości w następnym bloku z użyciem macierzy granicznej B . Łącznie operacje dla każdego bloku są ograniczone do $O(l^2)$, co dla wszystkich bloków daje złożoność czasową $O(v \cdot l^2) = O(n \cdot l)$ dla stałego, małego l .

Podstawienie wstecz

Analogicznie, podstawianie wsteczne po blokach wykorzystuje macierze U i korekty wynikające z bloków C . Każdy blok wymaga $O(l^2)$ operacji, więc całkowita złożoność dla wszystkich bloków wynosi $O(v \cdot l^2) = O(n \cdot l)$, przy czym pamięć jest zużywana wyłącznie na bloki diagonalne, graniczne i przekątne.

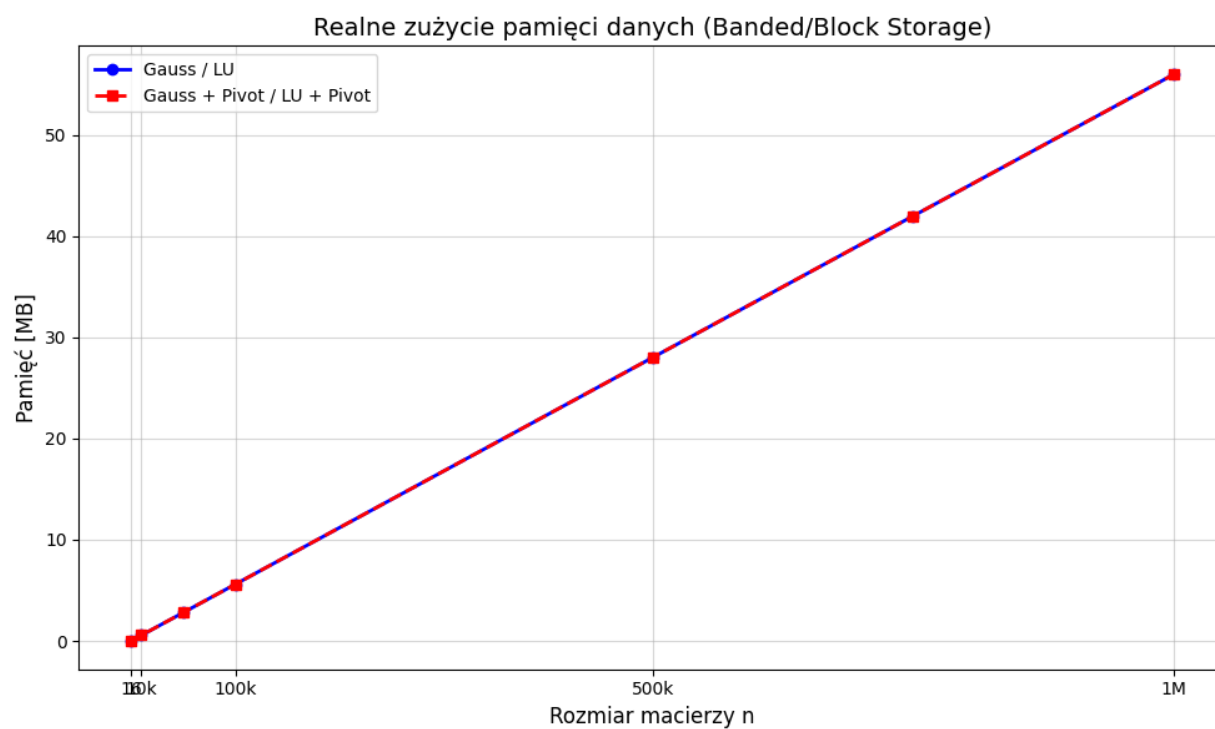
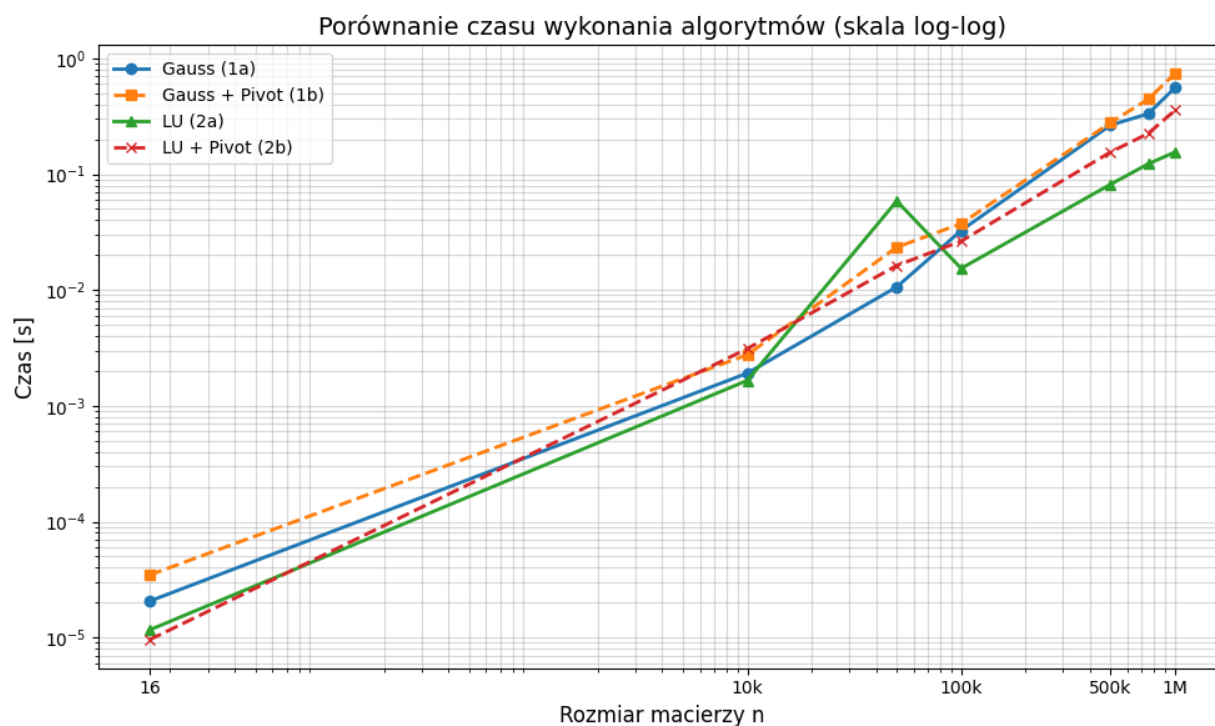
Idea metody i implementacja

Metoda wykorzystuje wcześniej obliczony blokowy rozkład LU . Rozwiązywanie układu $Ax = b$ sprowadza się do dwóch etapów: podstawiania w przód $Ly = Pb$ oraz podstawiania wstecz $Ux = y$. Dzięki strukturze blokowej w każdym kroku operujemy wyłącznie na elementach w bieżącym i sąsiednim bloku, co ogranicza liczbę operacji do $O(l^2)$ na blok i eliminuje konieczność przetwarzania całej macierzy. Permutacje p_k w blokach diagonalnych zapewniają stabilność numeryczną, a macierze B i C odpowiadają za wpływ sąsiednich bloków na wynik.

Algorithm 6 Rozwiązanie układu LU dla macierzy blokowej (Zadanie 3)

```
1: procedure SOLVELUBLOCK( $v, l, A, B, C, b, p$ )
2:    $x \leftarrow \text{copy}(b)$ 
3:   ▷ ETAP 1: Podstawienie w przód ( $Ly = Pb$ )
4:   for  $k \leftarrow 1$  to  $v$  do
5:      $r\_curr \leftarrow (k-1) \cdot l + 1 : k \cdot l$ 
6:     if  $k > 1$  then
7:        $r\_prev \leftarrow (k-2) \cdot l + 1 : (k-1) \cdot l$ 
8:        $z \leftarrow \text{copy}(x_{r\_prev})$ 
9:        $\text{Solve\_U}(A_{k-1}, z)$  ▷ Wykorzystanie  $U$  z poprzedniego bloku
10:       $x_{r\_curr[1]} \leftarrow x_{r\_curr[1]} - \text{dot}(B_{k-1}.\text{top\_row}, z)$ 
11:      for  $i \leftarrow 2$  to  $l$  do
12:         $x_{r\_curr[i]} \leftarrow x_{r\_curr[i]} - B_{k-1}.\text{right\_col}_{i-1} \cdot z_l$ 
13:      end for
14:    end if
15:     $\text{Solve\_L}(A_k, x_{r\_curr}, p_k)$  ▷ Rozwiązanie lokalne z permutacją  $p_k$ 
16:  end for
17:  ▷ ETAP 2: Podstawienie wsteczne ( $Ux = y$ )
18:   $\text{Solve\_U}(A_v, x_{(v-1)l+1:vl})$ 
19:  for  $k \leftarrow v-1$  down to  $1$  do
20:     $r\_k \leftarrow (k-1)l + 1 : kl$ 
21:     $r\_n \leftarrow kl + 1 : (k+1)l$ 
22:     $corr \leftarrow C_k.\text{diag} \cdot x_{r\_n}$  ▷ Mnożenie przez rzadki blok  $C$ 
23:     $\text{Solve\_L}(A_k, corr, p_k)$ 
24:     $x_{r\_k} \leftarrow x_{r\_k} - corr$ 
25:     $\text{Solve\_U}(A_k, x_{r\_k})$ 
26:  end for
27:  return  $x$ 
28: end procedure
```

Wykresy czasu i pamięci



Wnioski

Analiza wykresów czasu i pamięci potwierdza liniową złożoność obu implementacji względem rozmiaru macierzy n . Metoda z częściowym wyborem elementu głównego jest wolniejsza ze względu na dodatkowe operacje związane z wyszukiwaniem pivota i permutacjami, jednak zapewnia wyższą stabilność numeryczną. Wszystkie zaimplementowane warianty algorytmów wykazują identyczne, liniowe zużycie pamięci. Wynika to z zastosowania statycznej struktury blokowej, która alokuje optymalną ilość miejsca dla wszystkich elementów mogących stać się niezerowymi.